iDATA
Advanced analytics for business

PRESENTA A

BANCO DE DESARROLLO
DE AMÉRICA LATINA

DOCUMENTO DE ESTÁNDAR DE ARQUITECTURAS DE APLICACIONES

Camino El Alba 9500 Of 303. Santiago de Chile, Chile
Complejo Ruta N Calle 67 #52-20 Torre A INT 3015. Medellín, Colombia
www.idata.com.co @idata.co iDATA Global Analytics contacto@idata.com.co

Derechos de Autor y Confidencialidad del documento

IDATA S.A.S.
Derechos Reservados

El contenido de este documento y sus anexos se encuentran protegidos por el Derecho de Autor, y éstos son propiedad de IDATA S.A.S. y CAF S.A.S., por lo anterior, se le debe brindar tratamiento de confidencial. El cliente puede usar dicha información solo con el propósito de “apoyar la administración de la solución” sin poder divulgar su contenido a terceras partes ya que contiene ideas, conceptos y metodologías con propiedad de IDATA S.A.S. La clasificación "confidencial" significa que esta información es sólo para uso de las personas a quienes está dirigida. Se requiere de la autorización expresa y escrita de IDATA S.A.S. para realizar copias totales o parciales de este documento. Esta información no puede ser utilizada por el cliente para fines diferentes a los del objetivo del documento realizado por IDATA S.A.S.

Camino El Alba 9500 Of 303. Santiago de Chile, Chile
Complejo Ruta N Calle 67 #52-20 Torre A INT 3015. Medellín, Colombia
www.idata.com.co @idata.co iDATA Global Analytics contacto@idata.com.co

Versión	Responsable	Modificación	Fecha
V01	iData	Creación de Documento	18/03/2025

Camino El Alba 9500 Of 303. Santiago de Chile, Chile
Complejo Ruta N Calle 67 #52-20 Torre A INT 3015. Medellín, Colombia
www.idata.com.co @idata.co iDATA Global Analytics contacto@idata.com.co

Tabla de contenido

1. Introducción 6
2. Objetivo 6
2.1. Objetivo general. 6
2.2. Objetivos específicos 6
2.3. Referencias 6
3. Definiciones y abreviaciones 7
4. Descripción de la arquitectura en contexto 8
5. Roles y sus responsabilidades 8
6. Arquitecturas de soluciones 8
6.1. Resumen de las arquitecturas 8
6.2. Arquitectura del Escenario #1 8
6.2.1 Diseño de la arquitectura #1 8
6.2.2. Diagrama de arquitectura 9
6.2.3. Consideraciones de seguridad y funcionalidad 11
6.2.4. Costos asociados 11
6.3. Diseño de la arquitectura #2 12
6.3.1 Diseño de la arquitectura #2 12
6.3.2. Diagrama de arquitectura 13
6.3.3. Consideraciones de seguridad y funcionalidad 14
6.3.4. Costos asociados 15
6.4. Requerimientos no funcionales 16
6.4.1. Calidad 16
6.4.2. Cambios 16
6.4.3. Capacidad 16
6.4.4. Disponibilidad 16
6.4.5. Continuidad (diagnóstico y recuperación) 17
6.4.6. Consideraciones de seguridad 17
6.4.7. Requerimientos de ciberseguridad 17
6.5. Mapa de entrega continua 17
6.6. Repositorios GIT 18
6.7. Pipelines de CI/CD 18
6.8. Recomendaciones sobre la implementación de las arquitecturas 19
6.8.1. Mejoras generales 19
6.8.2. Consideraciones 21
6.9. Detalles de implementación de integraciones 21

6.9.1. Lineamientos para el desarrollo de software. 21
7. Referencias..... 22

Camino El Alba 9500 Of 303. Santiago de Chile, Chile
Complejo Ruta N Calle 67 #52-20 Torre A INT 3015. Medellín, Colombia
www.idata.com.co @idata.co iDATA Global Analytics contacto@idata.com.co

1. Introducción

La arquitectura de software, datos e integración define cómo las aplicaciones de negocio pueden ser reutilizadas, adaptadas y escaladas de manera eficiente para responder a las necesidades cambiantes de la organización. En un entorno empresarial en constante evolución, es fundamental contar con una arquitectura flexible que permita modificar funcionalidades y absorber el crecimiento sin comprometer la estabilidad ni la eficiencia operativa. En este sentido, el rápido crecimiento de una empresa exige una adaptación ágil de su arquitectura, servicios y sistemas de información para satisfacer la demanda de datos y garantizar la continuidad del negocio.

Este documento establece un estándar de arquitecturas de referencia para la implementación de soluciones en Azure. Su propósito es proporcionar un conjunto de lineamientos y modelos arquitectónicos que permitan a los equipos de tecnología seleccionar y diseñar soluciones óptimas de acuerdo con distintas necesidades empresariales. Cada arquitectura presentada en este documento incluye casos de uso, componentes clave, consideraciones técnicas y buenas prácticas que garantizarán implementaciones seguras, escalables y alineadas con las mejores prácticas en la nube.

2. Objetivo

2.1. Objetivo general.

Establecer un estándar de arquitecturas de referencia en Azure que sirva como guía para la implementación de soluciones tecnológicas escalables, seguras y eficientes. Este documento busca facilitar la toma de decisiones arquitectónicas, promoviendo la reutilización de componentes, la integración efectiva de sistemas y la optimización del rendimiento en entornos empresariales dinámicos.

2.2. Objetivos específicos.

- Definir arquitecturas base que aborden distintos casos de uso, proporcionando lineamientos claros sobre su implementación y mejores prácticas.
- Garantizar la escalabilidad y flexibilidad de las soluciones, permitiendo la adaptación de los sistemas a los cambios del negocio sin comprometer la estabilidad ni el rendimiento.
- Optimizar la integración de datos y servicios, asegurando la interoperabilidad entre sistemas mediante patrones y tecnologías adecuadas.
- Proveer criterios de selección de servicios de Azure, estableciendo cuándo y cómo utilizar cada componente en función de los requerimientos del negocio.
- Incorporar principios de seguridad y gobernanza, alineados con las mejores prácticas de Azure para la protección de datos, control de accesos y cumplimiento normativo.
- Facilitar la estandarización y reutilización de soluciones tecnológicas dentro de la organización, reduciendo costos y tiempos de desarrollo.

2.3. Referencias

- ISO 15489
- ACME
- ISO/IEC 9126: 2001

3. Definiciones y abreviaciones

Abreviatura	Definición
App Service	Plataforma de Azure que permite el despliegue y escalado de aplicaciones web, APIs y funciones en la nube sin necesidad de administrar infraestructura.
Azure Functions	Servicio de computación sin servidor que permite ejecutar código en respuesta a eventos sin gestionar infraestructura.
Service Bus	Mensajería empresarial confiable para la comunicación asincrónica entre aplicaciones y servicios distribuidos en la nube.
Log Analytics	Servicio de monitorización de Azure que recopila y analiza datos de logs de diversas fuentes para obtener información sobre el estado y rendimiento de los sistemas.
Azure Monitor	Plataforma de observabilidad que proporciona métricas, logs y alertas para supervisar el rendimiento y la disponibilidad de los recursos en Azure.
Key Vault	Servicio de Azure para la administración segura de secretos, claves de cifrado y certificados.
Storage Account	Servicio de almacenamiento en la nube de Azure que permite gestionar blobs, archivos, colas y tablas de datos.
Cosmos DB	Base de datos NoSQL distribuida globalmente, altamente escalable y diseñada para ofrecer alta disponibilidad y rendimiento.
Azure OpenAI	Servicio que proporciona acceso a modelos avanzados de inteligencia artificial, como GPT y DALL·E, permitiendo la integración de capacidades de procesamiento de lenguaje natural en aplicaciones empresariales.
Azure Databricks	Plataforma de análisis basada en Apache Spark que facilita la ingeniería de datos, el análisis avanzado y la ciencia de datos en entornos colaborativos y escalables.
Azure Document Intelligence	Servicio de IA que permite extraer información estructurada de documentos mediante modelos preentrenados y personalizados, facilitando la automatización del procesamiento de documentos.
Azure AD	Azure Active Directory es un servicio de administración de acceso e identidades basado en la nube.
GR	Un grupo de recursos es un contenedor que almacena los recursos relacionados con una solución de Azure.
Tenant	Un tenant o inquilino representa una organización. Se trata de una instancia dedicada de Azure AD que una organización o un desarrollador de aplicaciones recibe al principio de su relación con Microsoft.
LS	Los Linked Services son las conexiones (datos de conexión) para enlazar Azure Data Factory con servicios externos.
IR	Integration Runtime es una infraestructura (entorno de proceso) donde se gestionan las actividades de integración de datos.
ADF	Azure Data Factory es un servicio de integración de datos y ETL basado en la nube que le permite crear flujos de trabajo orientados a datos a fin de coordinar el movimiento y la transformación de datos a escala.
MVC	MVC es una propuesta de arquitectura del software utilizada para separar el código por sus distintas responsabilidades, manteniendo distintas capas que se encargan de hacer una tarea muy concreta.

4. Descripción de la arquitectura en contexto

La presente arquitectura se describe a través de múltiples referencias para llegar a un estándar que facilite su interpretación y aplicación al desarrollo de nuevos componentes, aplicaciones, microservicios, etc.

La arquitectura de aplicaciones describe una serie de patrones, estrategias y técnicas que se usan para diseñar y desarrollar las aplicaciones, servicios o integraciones. Esta arquitectura dicta unos lineamientos y proporciona un plan para garantizar el desarrollo de un componente bien definido.

5. Roles y sus responsabilidades

Rol	***Descripción***
Usuario	*Rol que consume los servicios expuestos.*
Desarrollador	*Rol encargado de diseñar y desarrollar las aplicaciones e integraciones de acuerdo con los lineamientos establecidos.*

6. Arquitecturas de soluciones

6.1. Resumen de las arquitecturas

Arquitecturas de aplicaciones que pretenden soportar y satisfacer con las necesidades de una entrega oportunidad, eficiente y precisa de la información requerida para diferentes escenarios contemplados. Además de establecer estándares y guías para la integración de nuevos componentes, reduciendo el riesgo a errores y facilitando los procesos de incorporación de nuevos servicios.

6.2. Arquitectura del Escenario #1

La arquitectura del escenario #1 se compone de varias capas que agrupan diferentes componentes que interactúan entre sí. El primer componente de estar arquitectura es el usuario quien realiza las distintas interacciones con un componente capaz de mostrar una interfaz web. El segundo componente general se refiera a todos los servicios desplegados en nube que en función de las necesidades es posible cambiarlos, eliminarlos o adaptarlos.

6.2.1 Diseño de la arquitectura #1

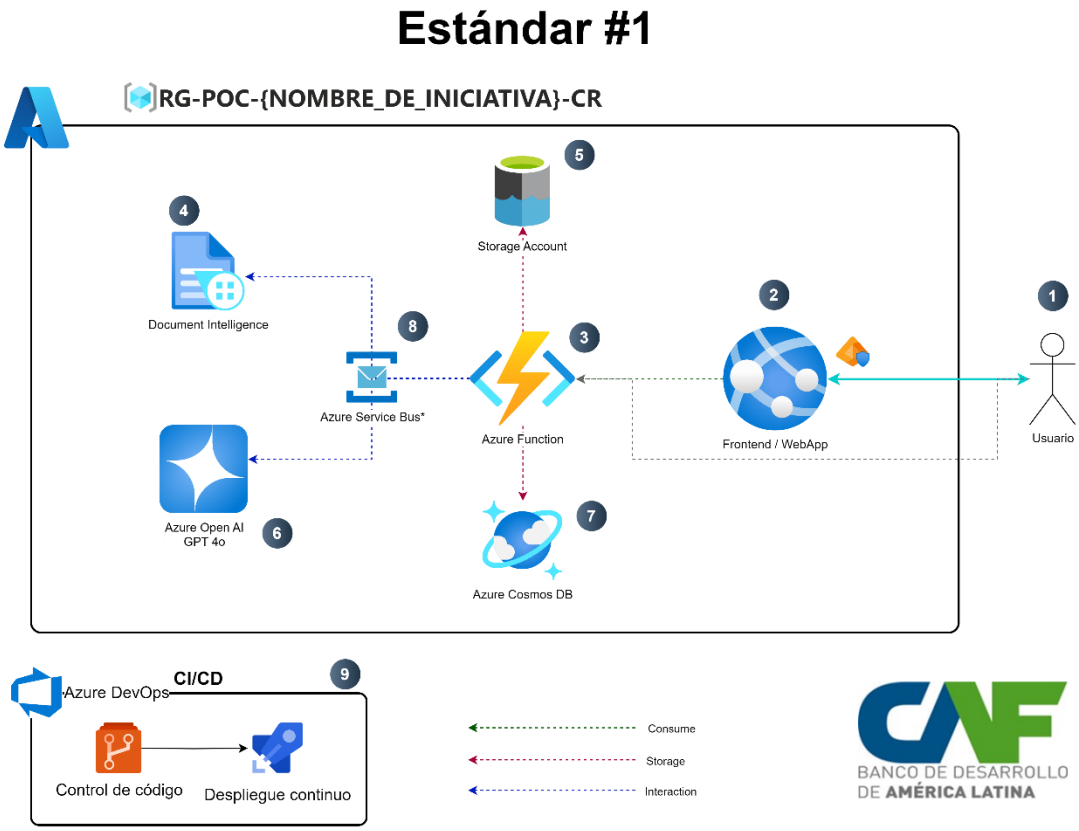
La arquitectura está diseñada y desplegada en la nube de Azure y cuenta con múltiples servicios entre ellos IA que se encargan de la identificación, extracción y almacenamiento de información de baja demanda pero con una respuesta en corto tiempo.

Escenarios de uso

Se aconseja adoptar esta arquitectura cuando la necesidad se base en la extracción de información de distintos tipos de documentos y que este proceso no se requiera realizar sobre múltiples documentos a la vez en tiempo real, dado que esta arquitectura no soporta trabajos de alta demanda y frecuencia. Es importante mencionar que si la información que se va a extraer proviene de "documentos plantilla" o formatos preestablecidos (es decir que el formato sea el mismo pero la información cambie) es posible prescindir del servicio Open IA y se recomendaría usar modelos custom pre entrenados de Document Intelligence. De no ser el caso mencionado anteriormente, Open AI será el servicio idóneo que realice la identificación y extracción de la información solicitada, y bajo el mismo escenario Document Intelligence se usará como servicio de OCR que entregará el texto a OpenIA.

Bajo esta arquitectura se puede simular una carga batch pero el tiempo de procesamiento podría ser mayor a algunas de las arquitecturas que se proponen más adelante. Para lograr esta carga batch se propone usar el servicio de Azure Service Bus quien se encargaría de realizar un encolamiento de las distintas peticiones realizadas e irlas gestionando a medida que se vayan liberando los procesos de extracción y almacenamiento de la información.

6.2.2. Diagrama de arquitectura



1. Usuarios funcionales quienes interactúan con la interfaz web (2) de la solución, cabe aclarar que si dicho usuario es técnico es posible que pueda interactuar directamente con las apis desplegadas en el servicio de Azure función(3).

2. Servicio que aloja la interfaz web que facilita el cargue de documentos y la visualización de los resultados en la extracción de la información. Es importante mencionar que es necesario realizar el desarrollo mediante código fuente de esta interfaz para posteriormente desplegarla en este servicio. La web app se encargará de enviar y recibir información del proceso realizado directamente desde y hacia las APIs disponibles en la Azure Function (3)
3. Este servicio tiene varias funciones pero dentro de estar arquitectura es utilizado para alojar todas las APIs que interactúan con los demás servicios (que se mencionan más adelante) para almacenar, procesar, extraer y retornar información dentro del proceso solicitado. Es importante resaltar que así como en el servicio del frontend (2), aquí también es necesario desarrollar las APIs mediante código, no existe una restricción en el lenguaje usado, sin embargo, se recomienda usar Python o C# por su facilidad de integración con los servicios desplegados en esta arquitectura.
4. Document Intelligence es el servicio por excelencia en la extracción y reconocimiento de información de múltiples documentos, cabe aclarar que su funcionamiento se basa en la identificación de la posición (coordenadas) de la información a extraer. Es por esto que si la necesidad se basa en extraer información sobre documentos que tengan un mismo formato, Document Intelligence es la mejor opción, y bajo este escenario se podría prescindir de Open IA (6). Sin embargo, de no ser este el escenario, Document Intelligence funcionaría como OCR, es decir se limitaría la extracción del texto del documento enviado, que sería usado después por Open IA (6).
5. La información extraída o el documento cargado será persistido dentro del servicio de Azure Storage Account, el cual así como Azure Functions (3) tiene múltiples usos, en este caso Azure Storage account puede ser usado como servicio de colas, servicio de tablas y de lago de datos. Pero en este escenario se usará para almacenar todos los documentos que se cargan para realizar la extracción, es posible prescindir de este servicio si por políticas de privacidad no se permite dejar huella de la información cargada.
6. Open IA es un componente clave en esta arquitectura dado que es quien realiza la identificación de las variables y la extracción de la información de los documentos o fuente de datos indicada. En este caso de uso es recomendable usar la versión GPT 4o del modelo de Open IA dado que es la que hasta ahora ha dado mejores resultados tanto en asertividad como en costos. Cabe resaltar que este servicio es prescindible de acuerdo a los escenarios mencionados anteriormente.
7. Ante la extracción es necesario contar con un servicio que garantice la persistencia de dicha información, es por esto por lo que Azure cosmos DB es el servicio utilizado para almacenar la información extraída con el fin de utilizarse más adelante. Es importante aclarar que si no es necesario almacenar la información extraída desde los servicios de Document Intelligence y/o Open IA se puede prescindir del servicio de Azure Cosmos DB.
8. Una de las características de esta arquitectura es que no es de alta demanda es decir, si envían 10 o 20 peticiones al tiempo, no se procesarán ni se responderán de manera simultánea. Estas peticiones estarán encoladas mediante el servicio de Azure Service Bus, quien se encarga de administrarlas, desencolando una a una. Es importante tener en cuenta que si las peticiones no serán muchas y no se necesita un encolamiento de estas, se puede prescindir de este servicio.

9. Por último, se recomienda manejar todo lo relacionado con el IaC y código fuente desarrollado dentro de Azure DevOps dado que nos permite realizar CI/CD y gestionar el ciclo de vida del proyecto de una manera ágil y transparente, en capítulos siguientes se hablará un poco más de este apartado.

6.2.3. Consideraciones de seguridad y funcionalidad

1. Se recomienda usar managed identity para realizar la comunicación entre los servicios desplegados, y desactivar el uso de shared keys.
2. Se almacenar el código fuente desarrollado en Azure DevOps y generar pipelines tanto de CI como CD para garantizar la automatización del despliegue.

6.2.4. Costos asociados

A continuación, se detallan los posibles costos asociados a cada arquitectura, es importante mencionar que este es un estimado y que puede variar en cualquier momento.

Service type	Description	Estimated monthly cost
Storage Accounts	Block Blob Storage, General Purpose V2, Flat Namespace, LRS Redundancy, Hot Access Tier, 1,000 GB Capacity - Pay as you go, 10 x 10,000 Write operations, 10 x 10,000 List and Create Container Operations, 10 x 10,000 Read operations, 1 x 10,000 Other operations. 1,000 GB Data Retrieval, 1,000 GB Data Write, SFTP disabled	$19,44
App Service	Standard Tier; 1 S3 (4 Core(s), 7 GB RAM, 50 GB Storage) x 730 Hours; Linux OS; 0 SNI SSL Connections; 0 IP SSL Connections; 0 Custom Domains; 0 Standard SLL Certificates; 0 Wildcard SSL Certificates	$277,40
Azure Functions	Premium tier, Pay as you go, EP2: 2 Cores(s), 7 GB RAM, 250 GB Storage, 1 Pre-warmed instances, 1 Additional scaled out units	$583,71
Azure AI Document Intelligence	Azure Form Recognizer, Commitment Tier, Azure standard, Custom: Custom pages - 20,000 pages commitment tier and 52,000 overage pages	$1.944,00
Azure OpenAI Service	Language Models, Standard (On-Demand), GPT-4o-Regional-API-128K, 60,000 x 1,000 input tokens, 0 x 1,000 output tokens	$300,00
Azure Cosmos DB	Azure Cosmos DB for NoSQL (formerly Core), Serverless, Always-free quantity disabled, Single Region Write (Single-Master) - East US (Write Region), 100 million RUs, 0 GB transactional storage, 2 copies of periodic backup storage, Dedicated Gateway not enabled	$25,00

Service Bus	Standard tier: Messaging Operations: 730 Hours of base charge, 0 x 1 million operations; 0 brokered connection(s); Hybrid Connections: 0 listener(s), 0 overage GB; WFC Relays: 0 x 100 relay hours, 0 x 10,000 message(s)	$9,81
Support		$0,00
Licensing Program	**Microsoft Customer Agreement (MCA)**	
Billing Account		
Billing Profile		
Total		**$3.159,36**

6.3. Diseño de la arquitectura #2

La arquitectura está diseñada y desplegada en la nube de Azure y cuenta con múltiples servicios entre ellos IA que se encargan de la identificación, extracción y almacenamiento de información de alta demanda y con una respuesta tardía.

6.3.1 Diseño de la arquitectura #2

La arquitectura está diseñada y desplegada en la nube de Azure y cuenta con múltiples servicios entre ellos IA que se encargan de la identificación, extracción y almacenamiento de información de baja demanda pero con una respuesta en corto tiempo.

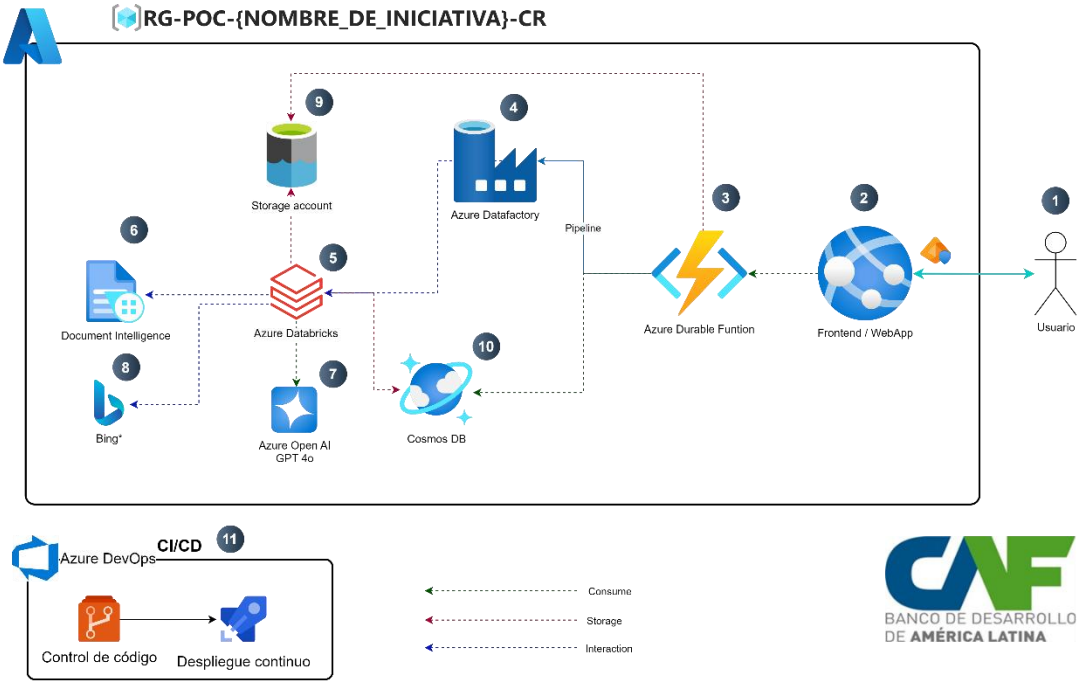
Escenarios de uso

Se aconseja adoptar esta arquitectura cuando la necesidad se base en la extracción de información de distintos tipos de documentos y que este proceso se requiera realizar sobre múltiples documentos a la vez en tiempo real, o que el volumen de extracción sea alto y demorado dado que esta arquitectura soporta trabajos de alta demanda y frecuencia haciéndolo ideal para soportar procesos que puedan tomar horas. Es importante mencionar que si la información que se va a extraer proviene de “documentos plantilla” o formatos preestablecidos (es decir que el formato sea el mismo pero la información cambie) es posible prescindir del servicio Open IA y se recomendaría usar modelos custom pre entrenados de Document Intelligence. De no ser el caso mencionado anteriormente, Open AI será el servicio idóneo que realice la identificación y extracción de la información solicitada, y bajo el mismo escenario Document Intelligence se usará como servicio de OCR que entregará el texto a OpenIA.

Bajo esta arquitectura se puede simular una carga batch de larga duración. Esto se puede interpretar de dos formas, la primera es la extracción de una cantidad grande de información sobre un solo documento y la segunda es la extracción de una cantidad no tan extensa de información pero sobre múltiples documentos. Para lograr esta carga batch se propone usar múltiples servicios especializados en cargas de alta demanda, los cuales se describen a continuación.

6.3.2. Diagrama de arquitectura

Estándar #2



1. Usuarios funcionales quienes interactúan con la interfaz web (2) de la solución, cabe aclarar que si dicho usuario es técnico es posible que pueda interactuar directamente con las apis desplegadas en el servicio de Azure función(3).
2. Servicio que aloja la interfaz web que facilita el cargue de documentos y la visualización de los resultados en la extracción de la información. Es importante mencionar que es necesario realizar el desarrollo mediante código fuente de esta interfaz para posteriormente desplegarla en este servicio. La web app se encargará de enviar y recibir información del proceso realizado directamente hacia y desde las APIs disponibles por la Azure Function (3)
3. Este servicio tiene varias funciones pero dentro de estar arquitectura es utilizado para alojar todas las APIs que interactúan con los demás servicios (que se mencionan a continuación) para almacenar, procesar, extraer y retornar información dentro del proceso solicitado. Es importante resaltar que así como en el servicio del frontend (2), aquí también es necesario desarrollar las APIs mediante código, no existe una restricción en el lenguaje usado, sin embargo, se recomienda usar Python o C# por su facilidad de integración con los servicios desplegados en esta arquitectura.
4. Azure DataFactory actúa como un orquestador dentro de esta arquitectura. Es el servicio encargado de recibir las peticiones de extracción que realiza el usuario y ejecutar una serie de pipelines que garantizan que el proceso no entre en un estado de timeout ni se pierda su ejecución sin importar su duración. Este servicio se conecta directamente con las Azure Functions (3), Azure Databricks (5) y Azure cosmos DB (6).
5. El desarrollo de notebooks garantiza la versatilidad en los distintos escenarios en los que sea necesario extraer información de documentos. Estos notebooks pueden ser desarrollados usando SQL o PySpark y ser ejecutados bajo de

manda o de manera programada por medio del servicio de Azure DataBricks. Este servicio añade una capa muy importante de personalización y ajuste sobre la necesidad de negocio. Además de ser altamente integrable con la mayoría de los servicios Azure.

6. Document Intelligence es el servicio por excelencia en la extracción y reconocimiento de información de múltiples documentos, cabe aclarar que su funcionamiento se basa en la identificación de la posición (coordenadas) de la información a extraer. Es por esto que si la necesidad se basa en extraer información sobre documentos que tengan un mismo formato, Document Intelligence es la mejor opción, y bajo este escenario se podría prescindir de Open IA (6). Sin embargo, de no ser este el escenario, Document Intelligence funcionaría como OCR, es decir se limitaría la extracción del texto del documento enviado, que sería usado después por Open IA (6).
7. Open IA es un componente clave en esta arquitectura dado que es quien realiza la identificación de las variables y la extracción de la información de los documentos o fuente de datos indicada. En este caso de uso es recomendable usar la versión GPT 4o del modelo de Open IA dado que es la que hasta ahora ha dado mejores resultados tanto en asertividad como en costos. Cabe resaltar que este servicio es imprescindible.
8. El servicio de Bing en Azure, a través de las APIs de Bing Search, permite a las aplicaciones acceder a capacidades avanzadas de búsqueda en la web. Estas APIs incluyen búsqueda de imágenes, videos, noticias, autosugestión y búsquedas personalizadas, aprovechando el motor de Bing para obtener información en tiempo real. Se pueden usar para mejorar chatbots, generar recomendaciones dinámicas, analizar tendencias o enriquecer aplicaciones con datos actualizados y relevantes desde internet. Este servicio no es imprescindible pero agrega mucho valor para enriquecer la información obtenida de la extracción del documento.
9. La información extraída o el documento cargado será persistido dentro del servicio de Azure Storage Account, el cual así como Azure Functions (3) tiene múltiples usos, en este caso Azure Storage account puede ser usado como servicio de colas, servicio de tablas y de lago de datos. Pero en este escenario se usará para almacenar todos los documentos que se cargan para realizar la extracción, es posible prescindir de este servicio si por políticas de privacidad no se permite dejar huella de la información cargada.
10. Ante la extracción es necesario contar con un servicio que garantice la persistencia de dicha información, es por esto por lo que Azure cosmos DB es el servicio utilizado para almacenar la información extraída con el fin de utilizarse más adelante. Es importante aclarar que si no es necesario almacenar la información extraída desde los servicios de Document Intelligence y/o Open IA se puede prescindir del servicio de Azure Cosmos DB.
11. Por último, se recomienda manejar todo lo relacionado con el IaC y código fuente desarrollado dentro de Azure DevOps dado que nos permite realizar CI/CD y gestionar el ciclo de vida del proyecto de una manera ágil y transparente, en capítulos siguientes se hablará un poco más de este apartado.

6.3.3. Consideraciones de seguridad y funcionalidad

1. Se recomienda usar managed identity para realizar la comunicación entre los servicios desplegados, y desactivar el uso de shared keys.

2. Se almacenar el código fuente desarrollado en Azure DevOps y generar pipelines tanto de CI como CD para garantizar la automatización del despliegue.

6.3.4. Costos asociados

A continuación, se detallan los posibles costos asociados a cada arquitectura, es importante mencionar que este es un estimado y que puede variar en cualquier momento.

Service type	Description	Estimated monthly cost
Storage Accounts	Block Blob Storage, General Purpose V2, Flat Namespace, LRS Redundancy, Hot Access Tier, 1,000 GB Capacity - Pay as you go, 10 x 10,000 Write operations, 10 x 10,000 List and Create Container Operations, 10 x 10,000 Read operations, 1 x 10,000 Other operations. 1,000 GB Data Retrieval, 1,000 GB Data Write, SFTP disabled	$19,44
App Service	Standard Tier; 1 S3 (4 Core(s), 7 GB RAM, 50 GB Storage) x 730 Hours; Linux OS; 0 SNI SSL Connections; 0 IP SSL Connections; 0 Custom Domains; 0 Standard SLL Certificates; 0 Wildcard SSL Certificates	$277,40
Azure Functions	Premium tier, Pay as you go, EP2: 2 Cores(s), 7 GB RAM, 250 GB Storage, 1 Pre-warmed instances, 1 Additional scaled out units	$583,71
Azure AI Document Intelligence	Azure Form Recognizer, Commitment Tier, Azure standard, Custom: Custom pages - 20,000 pages commitment tier and 52,000 overage pages	$1.944,00
Azure OpenAI Service	Language Models, Standard (On-Demand), GPT-4o-Regional-API-128K, 60,000 x 1,000 input tokens, 0 x 1,000 output tokens	$300,00
Azure Cosmos DB	Azure Cosmos DB for NoSQL (formerly Core), Serverless, Always-free quantity disabled, Single Region Write (Single-Master) - East US (Write Region), 100 million RUs, 0 GB transactional storage, 2 copies of periodic backup storage, Dedicated Gateway not enabled	$25,00
Azure Data Factory	Azure Data Factory V2 Type, Data Pipeline Service Type, Azure Integration Runtime: 100 Activity Run(s), 200 Data movement unit(s), 200 Pipeline activities, 200 Pipeline activities – External,	$151,05
Azure Databricks	All-Purpose Compute Workload, Premium Tier, 1 D32AV4 (32 vCPU(s), 128 GB RAM) x 730 Hours, Pay as you go, 6 DBU x 730 Hours	$3.530,28

Support		$0,00
Licensing Program	**Microsoft Customer Agreement (MCA)**	
Billing Account		
Billing Profile		
Total		**$6.830,88**

6.4. Requerimientos no funcionales

A continuación, se describen los elementos de valor no funcionales relevantes para la organización tal como la usabilidad, seguridad, rendimiento.

6.4.1. Calidad

Garantizar que la arquitectura desplegada cumpla con los estándares de calidad que proporcionen seguridad, usabilidad y otros elementos medibles en tiempo de ejecución. Manejar elementos para testear y escalar los elementos desarrollados.

6.4.2. Cambios

Asegurar el ciclo de vida para todos los cambios al ambiente productivo, que sean planificados, administrados y revisados en forma apropiada antes de su implementación y puesta en producción con el mínimo de interrupción en la prestación del servicio, todos los cambios deben estar soportados y orquestados con el uso de flujos en DevOps para realizar los diferentes reléase. Estos cambios realizados en ambientes de producción deberían suponer una interrupción del servicio no mayor a 1 minuto.

6.4.3. Capacidad

- Cada respuesta a las consultas realizadas por el usuario debe tardar en promedio menos de 3 segundos (Contando con una conectividad constante a los distintos servicios de internet)
- Las peticiones que necesiten el envío o recepción de un archivo deben utilizar un enlace que apunte a este para evitar la pérdida de rendimiento y aumento de costos.
- Los servicios de Azure deben tener la capacidad de recibir una cantidad alta de peticiones al mismo tiempo y ser capaces de soportarlas sin perder rendimiento.

6.4.4. Disponibilidad

- Los servicios desplegados en Azure deben tener una disponibilidad del 99,9% de las veces en que un usuario o aplicación externa intente accederlo.
- La probabilidad de falla del sistema no podrá ser mayor a 0,05.

Disponibilidad por servicio desplegado:

- Azure App Service: 99.95%.
- Azure Functions: 99.95%
- Azure Open IA: 99.99%
- Azure Data Factory: 99.99%
- Azure Data Bricks: 99.99%
- Azure Storage Account: 99.99%
- Azure Bing: 99.95%
- Azure Document Intelligence: 99.9%

- Azure Cosmos DB: 99.99%

6.4.5. Continuidad (diagnóstico y recuperación)

- La duración de las fallas lógicas no podrá ser mayor a 5 minutos, estas fallas en caso de ser responsabilidad de los desarrolladores tendrán que ser resueltas de manera efectiva y rápida, para garantizar la continuidad en el servicio.
- La duración de las fallas de red no podrá ser mayor a 10 minutos. Estas fallas están sujetas a la disponibilidad de los servicios en nube.
- Sujeto a los SLA de Microsoft Azure.
- La estrategia de recuperación está sujeta al diagnóstico de la falla por medio de las estrategias de control y detección de incidentes. Una vez identificada la falla, se diagnosticará la mejor forma de recuperar el funcionamiento normal de la arquitectura, ya sea por medio del despliegue de algunos de los componentes de la arquitectura o la ejecución de pipelines de integración continua.

6.4.6. Consideraciones de seguridad

Se mencionan algunos puntos para tener en cuenta:

- La comunicación entre las aplicaciones externas o usuarios se deben hacer por medio del servicio de Azure Active Directory. Lo cual obliga a que sean provistas de credenciales únicas.
- La comunicación entre servicios debería ser mediante Managed Identity.
- Se deben configurar políticas o integrar estrategias que permitan el bloqueo de peticiones a aplicaciones no autorizadas, es decir establecer los alcances que cierta aplicación puede tener dentro de los servicios.

6.4.7. Requerimientos de ciberseguridad

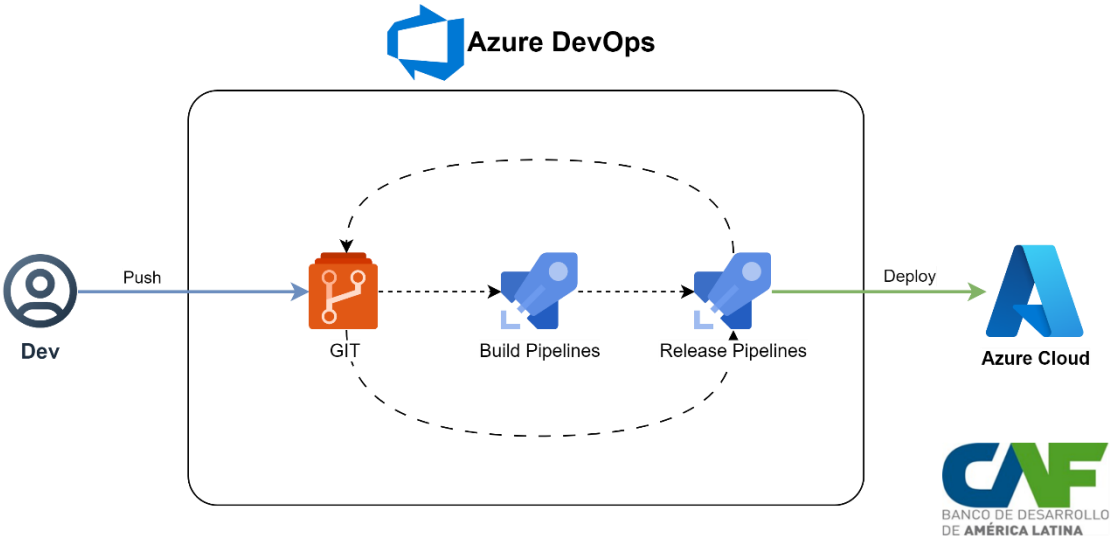
Con el fin de garantizar la seguridad en el acceso a los diferentes componentes y servicios expuestos en la arquitectura de plantean los siguientes requerimientos:

- Realizar el control sobre las aplicaciones que se dan de alta sobre el directorio activo.
- Realizar el cambio de credenciales para la autenticación sobre el directorio activo de manera periódica, se recomienda realizarlo de 1 a 2 años.
- Si es necesario el uso de un Service Principal, el secreto de este deberá cambiarse periódicamente.

6.5. Mapa de entrega continua

La integración y entrega continua se realiza a través de la plataforma de Microsoft Azure DevOps. Esta plataforma está integrada con GIT sobre la cual se carga todo el código fuente desarrollado de los servicios, así como de algunos otros servicios Azure de la arquitectura.

Por medio de la construcción de pipelines de compilación, el código cargado a los repositorios es compilado y alistado para que por medio de los pipelines de reléase se desplieguen directamente a los servicios Azure Cloud indicados. Este proceso es adaptable para cualquier ambiente, lo ideal es que este proceso empiece desde el ambiente de desarrollo, pase a un ambiente de calidad en donde se puedan realizar las validaciones necesarias y por último una vez confirmada la calidad del despliegue se exponga a los usuarios en el ambiente de producción.



6.6. Repositorios GIT

Azure Git será el repositorio central donde se depositará todo el código, las funcionalidades, mejoras de bugs, entre otros, con repositorios asignados para el ensamble de los servicios y alojados en Azure DevOps. Estos repositorios estarán alineados bajo los estándares de buenas prácticas, de manera que:

- Se usarán las ramas de máster, qa y dev, donde en máster reposará todo el código fuente de producción y no se deberá afectar directamente, la única forma de realizar cambios sobre la rama máster deberá ser a través de pull request generados desde la rama de QA. Lo anterior también aplica a la rama qa, actualizándose solamente cuando haya un pull request al ambiente de calidad que provenga del ambiente de desarrollo.
- Los nuevos desarrollos y soluciones de bugs deberán ser cargados en la rama de desarrollo.
- No se permitirá la interconexión entre ambientes, es decir, los proveedores deben contar con sus propios ambientes de desarrollo, pruebas y calidad y conectarse bajo los mismos ambientes.
- Debe existir una segregación de permisos sobre ambiente, es decir solamente deberá tener acceso sobre ambientes de calidad y productivo los usuarios estrictamente necesarios.

6.7. Pipelines de CI/CD

Se construirán los pipelines de CI/CD teniendo en cuenta las herramientas usadas para el desarrollo. Se plantean estos pasos genéricos para cada pipeline de CI, los cuales se modificarán teniendo en cuenta las necesidades técnicas que requiera cada microservicio:

Pasos básicos para el pipeline de CI: El agente de trabajo (Agent Job) será capaz de:

- Clonar el código fuente
- Instalación de librerías
- Ejecución de pruebas
- Publicación del artefacto

Pasos básicos para el pipeline de CD: teniendo en cuenta los ambientes de QA y producción definidos anteriormente, se debe construir un pipeline de CD capaz de tomar el artefacto creado previamente y promoverlo a los diferentes ambientes, según los resultados de las pruebas y los criterios de aceptación planteados:

- Desplegar a cada ambiente de servicio el artefacto previamente construido y aprobado.
- Crear o actualizar la documentación asociada a dicho microservicios.
- Generar reportes de errores asociadas al despliegue.

6.8. Recomendaciones sobre la implementación de las arquitecturas

6.8.1. Mejoras generales

6.8.1.1. Aplicaciones/Integraciones

Nombramiento de recursos y elementos dentro de la arquitectura

Con el fin de tener un mayor entendimiento sobre los recursos desplegados en la arquitectura de integraciones se propone adoptar una convención para el nombramiento de estos. Esta convención se basa en la que Microsoft propone, la cual establece lo siguiente:

rg-mdm-dev

Tipo de recurso – Aplicación – ambiente (se puede omitir para producción)

Para los elementos creados dentro del servicio Azure Data Factory, se recomienda utilizar la siguiente nomenclatura:

Type	Linked Service	Name	Linked Service	Dataset	Full
Azure	Blob storage	ABLB_	LS_ABLB_	DS_ABLB_	LS_ABLB_Example
	Data Lake Store	ADLS_	LS_ADLS_	DS_ADLS_	LS_ADLS_Example
	Azure SQL Database	ASQL_	LS_ASQL_	DS_ASQL_	LS_ASQL_Example
	Azure SQL DWH	ASDW_	LS_ASDW_	DS_ASDW_	LS_ASDW_Example
	Table storage	ATBL_	LS_ATBL_	DS_ATBL_	LS_ATBL_Example
	DocumentDB	ADOC_	LS_ADOC_	DS_ADOC_	LS_ADOC_Example
	Search Index	ASER_	LS_ASER_	DS_ASER_	LS_ASER_Example
DBs	SQL Server*	MSQL_	LS_SQL_	DS_SQL_	LS_SQL_Example
	Oracle*	ORAC_	LS_ORAC_	DS_ORAC_	LS_ORAC_Example
	MySQL*	MYSQ_	LS_MYSQ_	DS_MYSQ_	LS_MYSQ_Example
	DB2*	DB2_	LS_DB2_	DS_DB2_	LS_DB2_Example
	Teradata*	TDAT_	LS_TDAT_	DS_TDAT_	LS_TDAT_Example
	PostgreSQL*	POST_	LS_POST_	DS_POST_	LS_POST_Example
	Sybase*	SYBA_	LS_SYBA_	DS_SYBA_	LS_SYBA_Example
	Cassandra*	CASS_	LS_CASS_	DS_CASS_	LS_CASS_Example

	MongoDB*	MONG_	LS_MONG_	DS_MONG_	LS_MONG_Example
File	File System*	FILE_	LS_FILE_	DS_FILE_	LS_FILE_Example
	HDFS*	HDFS_	LS_HDFS_	DS_HDFS_	LS_HDFS_Example
	Amazon S3	AMS3_	LS_AMS3_	DS_AMS3_	LS_AMS3_Example
	FTP	FTP_	LS_FTP_	DS_FTP_	LS_FTP_Example
Others	Salesforce	SAFC_	LS_SAFC_	DS_SAFC_	LS_SAFC_Example
	Generic ODBC*	ODBC_	LS_ODBC_	DS_ODBC_	LS_ODBC_Example
	Generic OData	ODAT_	LS_ODAT_	DS_ODAT_	LS_ODAT_Example
	Web Table (table from HTML)	WEBT_	LS_WEBT_	DS_WEBT_	LS_WEBT_Example

Type	Name	Action	Example
Data movement Activity	PL_DATA_	NA	PL_DATA_DS_SQL_Person_To_DS_ABLB_Person
Data transformation pipeline	PL_TRAN_	SPRC – Stored Procedure	PL_TRAN_SPRC_CleanDimAccount
	PL_TRAN_	DNET – Script	PL_TRAN_DNET_AggregateSales
	PL_TRAN_	ADLK – Azure Data Lake	PL_TRAN_ADLK_AggregateSales
	PL_TRAN_	HIVE – Hive	PL_TRAN_HIVE_AggregateSales
	PL_TRAN_	PIG – Pig	PL_TRAN_PIG_AggregateSales
	PL_TRAN_	MAPR – MapReduce	PL_TRAN_MAPR_AggregateSales
	PL_TRAN_	HADP – Hadoop Stream	PL_TRAN_HADP_StreamData
	PL_TRAN_	AML – Azure Machine Learning	PL_TRAN_AML_CalculateMonthlyChurn

Nombramiento de elementos en DevOps

Es importante realizar un correcto nombramiento de los repositorios de código fuente de los distintos microservicios o APIs usadas en el negocio. Es por eso por lo que realizando la observación del estado actual de los repositorios que se tiene, se encuentra que:

- Es necesario actualizar el nombre de algunos repositorios usando el formato *CamelCase*. Es decir, sin usar espacios y usando mayúsculas en cada una de las palabras que contiene el nombre.
- Utilizar palabras que evidencien o faciliten el entendimiento del contenido del repositorio.

Hacer uso de ventajas ofrecidas por servicios desplegados

La nube de Microsoft ofrece múltiples ventajas en todos sus servicios. De acuerdo con la arquitectura desplegada se observa que se puede realizar la conexión con otros servicios que aporten mejoras en términos de diagnóstico, rendimiento y administración.

Se recomienda adoptar las siguientes sugerencias:

- Incluir el servicio de Application Insights para todos los servicios compatibles, este servicio puede proveer de métricas asociadas al rendimiento, errores, disponibilidad, uso, entre otras.

6.8.1.2. Seguridad

Migrar credenciales e información sensible al servicio Azure Key Vault

Es importante mencionar que la configuración de los Linked Services creados dentro de Azure Data Factory deben estar sobre Azure Key Vault.

6.8.2. Consideraciones

Para hacer efectiva la adaptación de la arquitectura actual a la arquitectura propuesta, es necesario tener en cuenta lo siguiente:

- Podría ser necesario realizar el redespliegue de ciertos servicios para renombrarlos de acuerdo con el estándar sugerido. Considerar si esta corrección no afecta la operación o no siguiere cambios traumáticos o retrocesos.
- El despliegue de nuevos servicios puede o no incurrir en costos adicionales a los que ya se tienen por la arquitectura actual.

6.9. Detalles de implementación de integraciones

6.9.1. Lineamientos para el desarrollo de software.

Con el fin de desarrollar software que siga una estructura definida que garantice la calidad, rendimiento y estandarización tanto del código fuente como de los elementos relacionados a la arquitectura, se definen los siguientes lineamientos:

Transversales

- El lenguaje por escoger para realizar el desarrollo de cualquier software (Móvil, Web, API o Desktop) debe seguir las indicaciones descritas en el punto 6.12.1. de este documento. Esto se debe hacer en función de la habilidad y experiencia de los desarrolladores en los lenguajes descritos.
- Para el nombramiento de los distintos componentes del desarrollo se deberá usar el siguiente formato:
    - Python: PEP 8 – Style Guide for Python Code | peps.python.org.
    - JavaScript: Google JavaScript Style Guide
- La estructuración del código fuente debería estar, en la medida de lo posible, guiado por el patrón de diseño MVC, el cual establece cierta jerarquía de carpetas en donde incorporar el código fuente desarrollado.
- El desarrollo debe realizarse con versiones de lenguajes de programación y framework recientes, no se deben usar versiones obsoletas. En caso de tener desarrollos en versiones sin soporte, se deberá hacer la respectiva actualización para evitar fallos por incompatibilidad en el futuro.

APIs o nuevas integraciones

- Para garantizar una estructura correcta del proyecto se debe seguir el patrón de diseño MVC (Model View, Controler) adaptado a la construcción de APIs y servicios. Esta adaptación es conocida como MIC (Model, Interface, Controller). Los archivos de código fuente creados deben repartirse en esta estructura de carpetas de la siguiente manera:
    - **Model:** En POO se ponen todas las clases identificadas.

  - **Inteface:** Archivos que contengan la construcción de las APIs y la conexión a las mismas.
  - **Controller:** Archivos que contengan toda la lógica del servicio y conexiones a fuentes de información.
- Todas las APIs desarrolladas deberán ser cargadas a un repositorio en Azure DevOps provisto por la organización, adicional a esto se deberán crear los respectivos pipelines de build y reléase dentro de la misma plataforma de Azure DevOps.
- El despliegue de estas APIs estará dirigido a una Azure Function nombrada bajo los estándares ilustrados en la sección Nombramiento de recursos y elementos dentro de la arquitectura de este documento.
- En caso de tener que intervenir una Azure Function existente, se deberá alterar el código fuente de la function en cuestión y realizar la ejecución de los pipelines de integración y despliegue continuo.
- Para el desarrollo de APIs debe estar bien definido e incorporado los estados HTTP para los diferentes errores (400, 404, 401, etc.). Además de esto, los mensajes de error que lleguen a los usuarios finales no deben contener información relacionada con bases de datos, sistemas operativos, red, lenguajes de programación, línea de error.
- Se deberá realizar la validación de entradas siempre y cuando se requieran, estas validaciones deben tratarse de modo que no contengan elementos maliciosos que puedan afectar o vulnerar la información del negocio.
- Los parámetros o variables que contengan información sensible como contraseñas o keys y que se envíen hacia una API deben ir en el body de la petición, a su vez esta API deberá aceptar únicamente peticiones POST.
- Se debe manejar el principio de **menor privilegio,** es decir, en la medida de lo posible se deberán usar:
  - Base de datos: Usuarios que tengan los privilegios restringidos y que sólo permita realizar tareas específicas sobre tablas o SPs específicos.
  - Microservicios: Usuarios que puedan acceder a ciertos microservicios específicos para sus necesidades.
  - Azure functions: Usar keys que permitan realizar tareas de consumo de APIs.

7. Referencias

- PageWriter-Msft. (s. f.). *Azure Well-Architected Framework - Microsoft Azure Well-Architected Framework*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/well-architected/
- *Zimmergren. (2025, 20 enero). Define your naming convention – Cloud Adoption Framework. Microsoft Learn. https://learn.microsoft.com/en us/azure/cloud-adoption-framework/ready/azure-best-practices/resource naming*
- Chcomley. (s. f.). *Azure DevOps documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/devops/?view=azure-devops